Guía de clase: CRUD de Customers con .NET 10 y EF Core

Proyecto: `CustomersApi` · **Duración:** 2 h 15 min · **Nivel:** inicial (C# básico)

1. Resumen ejecutivo

En 2 horas y cuarto, los alumnos recorren una API REST de clientes hecha con .NET 10, EF Core 10 y SQL Server, y salen sabiendo crear, aplicar y revertir migraciones por su cuenta.

- **Qué se construye:** un CRUD de `Customer` con 5 endpoints en `/api/customers` (listar, obtener, crear, actualizar y borrar), documentado con Swagger UI.
- **Cómo está organizado:** un solo proyecto con las capas en carpetas: `Controllers`, `Services`, `DataAccess` (DbContext, Configurations, Repositories), `Entities`, `Models` y `Mappers`.
- **El foco:** EF Core y migraciones. El proyecto trae 2 migraciones, `InitialCreate` y `AddCustomerPhone`, para mostrar cómo evoluciona una tabla y no solo cómo se crea.
- **Para quién:** desarrolladores que manejan C# básico (clases, interfaces, `async` / `await`) y nunca usaron EF Core, o lo usaron sin migraciones.
- **Formato:** recorrer el código terminado en lugar de escribirlo desde cero, y dedicar el tiempo en vivo a las migraciones y al ejercicio.
- **Resultado esperado:** cada alumno agrega la propiedad `BirthDate` al cliente, genera y aplica su migración y la prueba en Swagger.

2. Objetivos de aprendizaje

Al terminar la clase, el alumno puede:

1. Explicar qué hace cada capa (API, negocio y acceso a datos), por qué no se mezclan y en qué capa va cada validación.
2. Diferenciar una **entidad** (lo que se guarda) de un **DTO** (lo que viaja por HTTP).
3. Configurar una entidad con Fluent API: largos, campos requeridos, índice único, valor por defecto y datos semilla.
4. Crear, aplicar, listar y revertir migraciones con `dotnet ef`.
5. Leer una migración generada (`Up` y `Down`) y detectar un cambio que pierde datos.
6. Registrar dependencias con `AddScoped` y explicar por qué el DbContext vive lo que dura una petición.
7. Probar la API desde Swagger UI e interpretar los códigos 200, 201, 204, 400, 404 y 409.

3. Preparación antes de la clase

La mayoría de los problemas en una clase de EF Core vienen del entorno, no del código: deja todo probado el día anterior.

- ☐ SDK de .NET 10 instalado (`dotnet --list-sdks` muestra una versión 10.0.x). El `global.json` del proyecto fija el SDK 10 aunque haya un SDK 11 preview instalado.
- ☐ SQL Server Express o LocalDB funcionando, y la cadena de conexión de `CustomersApi/appsettings.json` apuntando a él.

- ☐ `dotnet tool restore` ejecutado en la carpeta raíz: instala `dotnet-ef` 10 como herramienta local.
- ☐ `dotnet build` termina con 0 errores y 0 advertencias.
- ☐ Base borrada para la demo: `dotnet ef database drop --force`. La clase arranca sin base, para que se vea cómo la crea la migración.
- ☐ Un visor de base abierto: SQL Server Management Studio o la extensión SQL Server (mssql) de VS Code.
- ☐ Swagger probado una vez en `http://localhost:5079/swagger`.
- ☐ Fuente grande en el editor y en la terminal (16 px o más) si vas a proyectar.
- ☐ Requisitos enviados a los alumnos con 2 días de anticipación: SDK 10, SQL Server Express o LocalDB, y Visual Studio o VS Code.

4. Agenda (2 h 15 min)

Hora	Bloque	Qué se ve	Archivos
0:00–0:10	1. Apertura	Demo de la API terminada en Swagger	`CustomersApi.http`, Swagger
0:10–0:30	2. Arquitectura en capas	Por qué separamos API, negocio y datos; recorrido de una petición	`Program.cs`, árbol de carpetas
0:30–0:45	3. Modelo y EF Core	Entidad, DbContext, Fluent API, índice único, seed	`Entities/`, `DataAccess/`
0:45–1:05	4. Migraciones en vivo	Crear la base, leer `Up` / `Down`, revertir	`Migrations/`, terminal, visor
1:05–1:10	Descanso		
1:10–1:30	5. Repositorio y servicio	Consultas, `SaveChanges`, regla de email único, DI	`DataAccess/Repositories/`, `Services/`
1:30–1:45	6. Controller y DTOs	Rutas, validación, mappers, códigos HTTP	`Controllers/`, `Models/`, `Mappers/`
1:45–2:05	7. Ejercicio práctico	Agregar `BirthDate` con su migración	todos
2:05–2:15	8. Cierre	Preguntas de repaso, tarea, próximos pasos	

Las migraciones van antes que el repositorio a propósito: primero existe la tabla, después el código que la usa.

5. Arquitectura de la API: por qué separamos en capas

Dividimos la API en tres capas (API, negocio y acceso a datos) para que cada parte del código tenga una sola razón para cambiar. Así, un cambio en el formato del JSON, en una regla de negocio o en la base de datos toca una sola carpeta.

La analogía del restaurante

Sirve para abrir el tema antes de mostrar código:

En un restaurante	En la API	Carpeta
El mozo toma el pedido y lleva el plato a la mesa	Controller	`Controllers/`
El chef decide si el plato se puede preparar y cómo	Servicio	`Services/`
El encargado de la despensa sabe dónde está cada ingrediente	Repositorio	`DataAccess/Repositories/`
La despensa	Base de datos	SQL Server

El cliente del restaurante no entra a la cocina, y el chef no atiende mesas. Si cambia el proveedor de verduras, el menú sigue igual.

El problema: todo en el controller

Sin capas, crear un cliente se vería así:

```
[HttpPost]
public async Task<IActionResult> PostCustomer(CustomerRequest request)
{
    // HTTP, regla de negocio y acceso a datos mezclados en un solo método
    if (await _dbContext.Customers.AnyAsync(c => c.Email == request.Email))
        return Conflict("Email repetido");

    var customer = new Customer
    {
        FirstName = request.FirstName,
        LastName = request.LastName,
        Email = request.Email,
        Phone = request.Phone
    };

    _dbContext.Customers.Add(customer);
    await _dbContext.SaveChangesAsync();

    return CreatedAtAction(nameof(GetCustomer), new { id = customer.Id }, customer);
}
```

Funciona, y para un CRUD de juguete alcanza. Los problemas aparecen cuando el proyecto crece:

- **La regla se copia.** El `PUT` necesita la misma verificación de email, y también la necesitaría un proceso que importe clientes desde un Excel. Cada copia puede quedar distinta.
- **No se puede probar la regla sola.** Para verificar "no se repite el email" hace falta levantar HTTP y una base de datos.

- **Se expone la entidad.** El método devuelve `Customer` tal cual: cualquier propiedad interna que se agregue mañana sale en el JSON.
- **Cambiar la base toca todo.** Cada controller usa EF Core directamente.

Qué hace cada capa

Capa	Carpetas	Responsabilidad	Conoce	No conoce
API	`Controllers/`, `Models/`, `Mappers/`	Recibir HTTP, validar el formato, mapear DTO ↔ entidad y elegir el código de respuesta	HTTP, DTOs, el servicio	EF Core, SQL, las reglas
Negocio	`Services/`	Decidir qué se puede hacer: las reglas	Entidades, el repositorio	HTTP, DTOs, EF Core
Acceso a datos	`DataAccess/`	Leer y guardar entidades	EF Core, el DbContext, LINQ	HTTP, las reglas
Dominio	`Entities/`	Representar los datos	Nada	Todo lo demás

`Entities/` no es una capa del recorrido: es el "idioma" que comparten negocio y acceso a datos.

El recorrido de una petición

Una petición atraviesa las tres capas y llega a SQL Server a través del `DbContext` de EF Core. La respuesta hace el camino inverso: entidad → `CustomerResponse` → JSON.

```
Cliente HTTP  (Swagger UI o CustomersApi.http)
   │  JSON → CustomerRequest
   ▼
Controllers/CustomersController.cs ........ valida el DTO, lo mapea y traduce errores a HTTP
   │  entidad Customer
   ▼
Services/CustomersService.cs .............. reglas de negocio: el email no se repite
   │  Customer ya validado
   ▼
DataAccess/Repositories/CustomersRepository.cs ... la única capa que usa EF Core
   │  consultas LINQ
   ▼
DataAccess/CustomersDbContext.cs .......... traduce LINQ a SQL y guarda con SaveChanges
   │  SQL (INSERT, SELECT, UPDATE, DELETE)
   ▼
SQL Server: base CustomersApiDb, tabla Customers
```

`Program.cs` registra cada capa con `AddScoped`: ASP.NET Core crea las instancias y se las inyecta a la capa de arriba, una por petición.

Las reglas que mantienen separadas las capas

1. **Las dependencias van en un solo sentido:** API → negocio → acceso a datos. El repositorio nunca llama al servicio, y el servicio nunca llama al controller.
2. **Cada capa usa la de abajo a través de una interfaz.** El controller pide un `ICustomersService` y el servicio pide un `ICustomersRepository`; `Program.cs` decide qué clase concreta se entrega.

3. **Los DTOs no pasan de la API, y las entidades no salen por HTTP.** El mapper es la frontera entre las dos cosas.
4. **El negocio informa errores sin hablar HTTP.** El servicio lanza `EntityNotFoundException` o `DuplicateEmailException`, y el controller las traduce a 404 y 409. El servicio no sabe qué es un 404.

Dónde vive cada validación

Validación	Capa	Ejemplo en el proyecto	Si falla
Formato del dato	API	`[Required]` y `[EmailAddress]` en `CustomerRequest`	400, automático por `[ApiController]`
Regla de negocio	Negocio	Email no repetido, en `EnsureEmailIsAvailableAsync`	`DuplicateEmailException` → 409
Integridad	Base de datos	Índice único `IX_Customers_Email`	Excepción de SQL Server: es la última red, no debería llegar

Qué ganamos y qué cuesta

Lo que ganamos, con ejemplos de este proyecto:

- **Cambios aislados.** Pasar a PostgreSQL toca `DataAccess/`, `Program.cs` y las migraciones. Cambiar el JSON toca `Models/` y `Mappers/`. Una regla nueva toca `Services/`.
- **Reutilización.** Un proceso que importe clientes desde un Excel usaría `ICustomersService`, y la regla del email se cumpliría sin copiarla.
- **Pruebas.** El servicio se prueba con un repositorio falso que implemente `ICustomersRepository`, sin base ni HTTP.
- **Lectura y trabajo en equipo.** Cada archivo es corto y tiene un propósito; dos personas trabajan en capas distintas sin pisarse.

Lo que cuesta: más archivos y métodos que solo pasan la llamada, como `GetAllAsync` del servicio, que solo llama al repositorio. Es normal en un CRUD simple: la capa de negocio gana valor a medida que aparecen reglas. Vale aclarar en clase que el `DbContext` de EF Core ya funciona como un repositorio, y que muchos equipos lo usan directamente desde el servicio. Aquí agregamos el repositorio para que EF Core quede encerrado en una sola carpeta y se vea cada responsabilidad por separado.

Errores típicos al separar en capas

- Poner reglas de negocio en el controller, por ejemplo verificar el email ahí.
- Que el controller use el repositorio o el `DbContext` directamente y se saltee el servicio.
- Que el servicio reciba DTOs o devuelva `IActionResult`: queda atado a HTTP y no se puede reutilizar.
- Que el repositorio decida reglas, por ejemplo que lance "email duplicado".
- Devolver entidades en lugar de DTOs.

Y las migraciones

Las migraciones recorren otro camino, desde la terminal y antes de cualquier petición:

```
Cambias Entities/ o DataAccess/Configurations/
   │  dotnet ef migrations add <Nombre>    (compara el modelo con el snapshot)
   ▼
Migrations/<fecha>_<Nombre>.cs            (métodos Up y Down)
   │  dotnet ef database update
   ▼
SQL Server: cambia el esquema y anota la migración en __EFMigrationsHistory
```

6. Conceptos clave

Concepto	Qué es	Dónde verlo
Entidad	Clase C# que representa una fila de una tabla.	`Entities/Customer.cs`
DTO	Objeto que entra o sale por HTTP; separa el contrato de la API del modelo de base.	`Models/CustomerRequest.cs`, `Models/CustomerResponse.cs`
Mapper	Convierte DTO ↔ entidad.	`Mappers/CustomerMapper.cs`
DbContext	Sesión con la base: consulta, rastrea cambios y guarda.	`DataAccess/CustomersDbContext.cs`
DbSet	Una tabla vista como colección que se consulta con LINQ.	`DbSet<Customer> Customers`
Fluent API	Configuración del mapeo (largos, requeridos, índices) fuera de la entidad.	`DataAccess/Configurations/CustomerConfiguration.cs`
Seed (`HasData`)	Datos iniciales que se insertan con una migración.	`CustomerConfiguration.cs`
Migración	Cambio de esquema escrito en C#: `Up` lo aplica y `Down` lo revierte.	`Migrations/*_InitialCreate.cs`
Model snapshot	Foto del modelo actual; EF la compara con tu código para generar la próxima migración.	`Migrations/CustomersDbContextModelSnapshot.cs`
`__EFMigrationsHistory`	Tabla de la base que registra qué migraciones se aplicaron.	Visor de base
Change tracking	EF recuerda las entidades que cargó y, en `SaveChanges`, genera los INSERT, UPDATE y DELETE que correspondan.	`CustomersRepository.cs`
`AsNoTracking`	Consulta de solo lectura, más liviana, sin rastreo de cambios.	`CustomersRepository.GetAllAsync`
Repositorio	La única capa que usa EF Core.	`DataAccess/Repositories/`
Servicio	Reglas de negocio; no conoce HTTP ni EF Core.	`Services/CustomersService.cs`
Inyección de dependencias	ASP.NET Core crea y entrega las dependencias; `AddScoped` = una instancia por petición.	`Program.cs`
`[ApiController]`	Valida el DTO y devuelve 400 automáticamente si falla.	`Controllers/CustomersController.cs`

Concepto	Qué es	Dónde verlo
OpenAPI / Swagger UI	Documento que describe la API / página web para probarla.	`/openapi/v1.json`, `/swagger`

7. Guion bloque por bloque

Bloque 1 · Apertura (10 min)

- Muestra Swagger: listar, crear un cliente, editarlo, borrarlo y un POST con un email repetido (409).
- **Frase clave:** "Hoy no vamos a memorizar código: vamos a entender por dónde pasa una petición y cómo la base sigue al código."
- **Pregunta:** ¿Dónde se guardó el cliente que acabo de crear? ¿Quién creó esa tabla?

Bloque 2 · Arquitectura en capas (20 min)

- Abre con la analogía del restaurante (sección 5).
- Muestra el "controller que hace todo" y pregunta qué habría que copiar para importar clientes desde un Excel.
- Sigue un POST abriendo los archivos en orden: Controller → Service → Repository → DbContext.
- En `Program.cs`, muestra los `AddScoped` y remarca que nadie escribe `new CustomersService(...)`.
- Cierra con la tabla "Dónde vive cada validación": tres validaciones de email, cada una en su capa.
- **Frase clave:** "Cada capa tiene una sola razón para cambiar: HTTP, reglas de negocio o acceso a datos."
- **Pregunta:** ¿Por qué el servicio lanza una excepción en lugar de devolver un 404?
  - *Respuesta:* porque el servicio no sabe de HTTP. Si mañana lo usa un proceso de importación, un 404 no significa nada ahí.
- **Pregunta:** Si mañana cambiamos SQL Server por PostgreSQL, ¿qué tocamos?
  - *Respuesta:* el paquete y el proveedor (`UseSqlServer`), la cadena de conexión, lo propio de SQL Server en la configuración (como `GETUTCDATE()`) y las migraciones. Controllers y servicios no cambian.

Bloque 3 · Modelo y EF Core (15 min)

- `Customer.cs`: propiedades simples; por convención, `Id` es la clave primaria.
- `CustomersDbContext.cs`: el `DbSet` es la tabla; `OnModelCreating` aplica la configuración.
- `CustomerConfiguration.cs`, línea por línea:
  - `IsRequired()` → columna `NOT NULL`.
  - `HasMaxLength(100)` → `nvarchar(100)`.
  - `HasIndex(...).IsUnique()` → índice único `IX_Customers_Email`.
  - `HasDefaultValueSql("GETUTCDATE()")` → SQL Server completa `CreatedAt`.
  - `HasData(...)` → 3 clientes semilla.
- **Frase clave:** "El modelo de C# es la fuente de verdad; la base se deriva de él."
- **Pregunta:** ¿Por qué el seed usa una fecha fija y no `DateTime.Now`?
  - *Respuesta:* porque si el valor cambia en cada ejecución, EF ve el modelo distinto y quiere generar una migración nueva cada vez.

Bloque 4 · Migraciones en vivo (20 min)

- Sigue el guion de la sección 8.

- **Frase clave:** "Una migración es como un commit del esquema: se crea, se revisa, se aplica y no se reescribe."
- **Pregunta:** Si borro la carpeta `Migrations`, ¿qué pierdo?
  - *Respuesta:* el historial y la posibilidad de actualizar paso a paso una base que ya existe.

Bloque 5 · Repositorio y servicio (20 min)

- `CustomersRepository`: `AsNoTracking` en la lista, `FindAsync` por clave, y `Add`, `Update` y `Remove` seguidos de `SaveChangesAsync`.
- Muestra que, después de `SaveChangesAsync`, el cliente nuevo ya tiene el `Id` y el `CreatedAt` que generó SQL Server.
- `CustomersService`: la regla del email único vive acá, no en el controller ni en el repositorio.
- **Frase clave:** "El repositorio sabe cómo guardar; el servicio decide si se puede."
- **Pregunta:** El índice único ya impide emails repetidos. ¿Para qué validar en el servicio?
  - *Respuesta:* para devolver un 409 con un mensaje claro en lugar de una excepción de SQL Server convertida en error 500.

Bloque 6 · Controller y DTOs (15 min)

- `[ApiController]` valida `CustomerRequest` y responde 400 sin código extra.
- Por qué no exponemos la entidad: quien llama a la API no debe poder enviar `Id` ni `CreatedAt`.
- Traducción de excepciones: `EntityNotFoundException` → 404, `DuplicateEmailException` → 409.
- `CreatedAtAction` → 201 con el header `Location` apuntando al cliente nuevo.
- Ejecuta los casos de error de `CustomersApi.http`.
- **Frase clave:** "El controller traduce: de HTTP al dominio y del dominio a HTTP."
- **Pregunta:** ¿Qué pasa si agrego una propiedad a la entidad y me olvido del DTO?
  - *Respuesta:* la columna existe en la base, pero la API no la recibe ni la devuelve.

8. Demo en vivo de migraciones

Es el momento central de la clase. Proyecta la terminal y el visor de base lado a lado y ejecuta cada comando desde `CustomersApi/CustomersApi`. Detén la API antes de cada `dotnet ef`, porque el comando compila el proyecto.

1. **Partir sin base.** `dotnet ef database drop --force`. En el visor, `CustomersApiDb` ya no existe.
2. **Ver qué hay pendiente.** `dotnet ef migrations list` muestra las 2 migraciones con `(Pending)`.
3. **Leer antes de aplicar.** Abre `Migrations/*_InitialCreate.cs`. En `Up` hay `CreateTable`, `InsertData` y `CreateIndex`; en `Down`, `DropTable`.
4. **Aplicar solo la primera.** `dotnet ef database update InitialCreate`. En el visor aparece la tabla `Customers` sin `Phone`, con 3 filas, y `__EFMigrationsHistory` con 1 fila.
5. **Aplicar el resto.** `dotnet ef database update`. Aparece la columna `Phone` y una segunda fila en el historial.
6. **Cargar un dato.** Levanta la API, crea un cliente con teléfono desde Swagger y vuelve a detenerla.
7. **Revertir.** `dotnet ef database update InitialCreate`. Se ejecuta el `Down` de `AddCustomerPhone`: la columna desaparece con sus datos.
8. **Volver a aplicar.** `dotnet ef database update`. La columna vuelve, pero vacía: el teléfono cargado en el paso 6 se perdió. Por eso EF avisa "*An operation was scaffolded that may result in the loss of data*".
9. **Ver el SQL para producción.** `dotnet ef migrations script --idempotent -o db.sql`. Cada bloque se protege con `IF NOT EXISTS`, así que el script se puede ejecutar sobre cualquier versión de la base.

10. **Opcional: el snapshot en acción.** `dotnet ef migrations add Prueba` sin cambiar nada genera `Up` y `Down` vacíos. Bórrala con `dotnet ef migrations remove`.

Para...	CLI	Visual Studio (Package Manager Console)
Crear una migración	`dotnet ef migrations add Nombre`	`Add-Migration Nombre`
Aplicar las pendientes	`dotnet ef database update`	`Update-Database`
Listar migraciones	`dotnet ef migrations list`	`Get-Migration`
Volver a una anterior	`dotnet ef database update InitialCreate`	`Update-Database InitialCreate`
Borrar la última (no aplicada)	`dotnet ef migrations remove`	`Remove-Migration`
Generar script SQL	`dotnet ef migrations script --idempotent`	`Script-Migration -Idempotent`
Borrar la base	`dotnet ef database drop`	`Drop-Database`

9. Ejercicio práctico: agregar la fecha de nacimiento

Enunciado (20 min): agrega al cliente una fecha de nacimiento opcional, `BirthDate`, de punta a punta: base, API y Swagger.

Pasos:

1. Agrega la propiedad a la entidad.
2. Genera la migración `AddCustomerBirthDate` y revísala.
3. Aplícala a la base.
4. Agrega la propiedad a los DTOs, al mapper y al `UpdateAsync` del servicio.
5. Prueba en Swagger: crea un cliente con `"birthDate": "1990-05-20"`, edítalo y consúltalo.

Criterios de éxito:

- ☐ La tabla `Customers` tiene la columna `BirthDate` de tipo `date`, que acepta nulos.
- ☐ `__EFMigrationsHistory` tiene 3 filas.
- ☐ `GET /api/customers/{id}` devuelve `birthDate`.
- ☐ `PUT` cambia la fecha.

Desafío extra: crea un atributo de validación `[NotInFuture]` que rechace fechas futuras con un 400 (MiPrimerApi tiene ejemplos de atributos propios en `Validations/`).

Solución

`Entities/Customer.cs`, `Models/CustomerRequest.cs` y `Models/CustomerResponse.cs`:

```
public DateOnly? BirthDate { get; set; }
```

```
dotnet ef migrations add AddCustomerBirthDate
dotnet ef database update
```

La migración generada agrega la columna y además actualiza las 3 filas semilla, porque `HasData` las registra en el snapshot:

```
migrationBuilder.AddColumn<DateOnly>(
    name: "BirthDate",
    table: "Customers",
    type: "date",
    nullable: true);

migrationBuilder.UpdateData(
    table: "Customers",
    keyColumn: "Id",
    keyValue: 1,
    column: "BirthDate",
    value: null);
// ... lo mismo para los Id 2 y 3
```

`Mappers/CustomerMapper.cs`:

```
// En ToEntity
BirthDate = request.BirthDate

// En ToDto
BirthDate = customer.BirthDate
```

`Services/CustomersService.cs`, dentro de `UpdateAsync`:

```
existingCustomer.BirthDate = customer.BirthDate;
```

10. Errores comunes

Síntoma	Causa	Solución
`dotnet ef` falla con *Build failed*: el archivo está bloqueado por `CustomersApi`	La API sigue corriendo y bloquea lo que se compila	Detener la API (Ctrl+C o Stop en Visual Studio) y repetir
The model for context 'CustomersDbContext' has pending changes. Add a new migration before updating the database.	Se cambió una entidad o una configuración sin crear la migración	`dotnet ef migrations add <Nombre>` y después `dotnet ef database update`
`dotnet ef` no existe, o avisa que la versión de las herramientas es anterior a la del runtime	Falta la herramienta local, o se usa una global vieja	`dotnet tool restore` en la carpeta raíz del proyecto
A network-related or instance-specific error al conectar	SQL Server apagado, o un nombre de instancia distinto	Revisar el servicio de SQL Server o cambiar la cadena a LocalDB
La propiedad nueva está en la base pero no en Swagger	Se agregó a la entidad y no al DTO ni al mapper	Completar `CustomerRequest`, `CustomerResponse` y `CustomerMapper`

Síntoma	Causa	Solución
Cada `migrations add` genera `UpdateData` aunque no cambie nada	Seed con valores dinámicos (`DateTime.Now`, `Guid.NewGuid()`)	Usar valores fijos en `HasData`
`migrations remove` dice que la migración ya fue aplicada	No se puede borrar una migración aplicada	`dotnet ef database update <Anterior>` y después `migrations remove`
Se editó una migración ya aplicada y la base no cambia	EF solo mira el nombre en `__EFMigrationsHistory`, no el contenido	Crear una migración nueva con la corrección
La plantilla crea un proyecto `net11.0`	Hay un SDK preview instalado y no hay `global.json`	Usar el `global.json` que fija el SDK 10

Cadena de conexión para LocalDB:

```
Server=
(localdb)\MSSQLLocalDB;Database=CustomersApiDb;Trusted_Connection=True;TrustServerCertificate=True;
```

En `appsettings.json`, la barra invertida va doble: `(localdb)\\MSSQLLocalDB`.

11. Cierre

Preguntas de repaso

Pregunta	Respuesta esperada
¿Qué diferencia hay entre una entidad y un DTO?	La entidad se guarda en la base; el DTO es lo que entra o sale por HTTP.
¿Qué archivo usa EF para saber qué cambió desde la última migración?	`CustomersDbContextModelSnapshot.cs`
¿Cómo vuelvo la base a una migración anterior y qué método se ejecuta?	`dotnet ef database update <Nombre>`; se ejecuta el `Down` de las posteriores.
¿Por qué el DbContext se registra como Scoped?	Para tener una instancia por petición: no se comparte entre usuarios y se libera al terminar.
¿En qué capa va "no puede haber dos clientes con el mismo email"?	En el servicio, porque es una regla de negocio.
¿Por qué el controller no usa el repositorio directamente?	Porque se saltearía las reglas del servicio, y habría que copiarlas en cada controller.

Tarea

- ☐ Agregar `City` (texto opcional, máximo 100 caracteres) de punta a punta, con la migración `AddCustomerCity`.
- ☐ Agregar un filtro `GET /api/customers?search=tur` que busque por nombre o apellido; el `Where` va en el repositorio.
- ☐ Generar el script idempotente y explicar en 3 líneas por qué se usa en producción en lugar de `database update`.

Próximos pasos

Para una segunda clase: una relación 1 a N (`Customer` → `Orders`) con su migración, manejo global de errores con `ProblemDetails` y validaciones con FluentValidation, como en MiPrimerApi.